Solution: Minimum Deletions to Make a Sequence Sorted

Problem Statement

Given a number sequence, find the minimum number of elements that should be deleted to make the remaining sequence sorted.

Example 1:

```
Input: {4,2,3,6,10,1,12}
Output: 2
Explanation: We need to delete {4,1} to make the remaing sequence sorted
{2,3,6,10,12}.
```

Example 2:

```
Input: {-4,10,3,7,15}
Output: 1
Explanation: We need to delete {10} to make the remaing sequence sorted
{-4,3,7,15}.
```

Example 3:

```
Input: {3,2,1,0}
Output: 3
Explanation: Since the elements are in reverse order, we have to delete
all except one to get a
sorted sequence. Sorted sequences are {3}, {2}, {1}, and {0}
```

Basic Solution

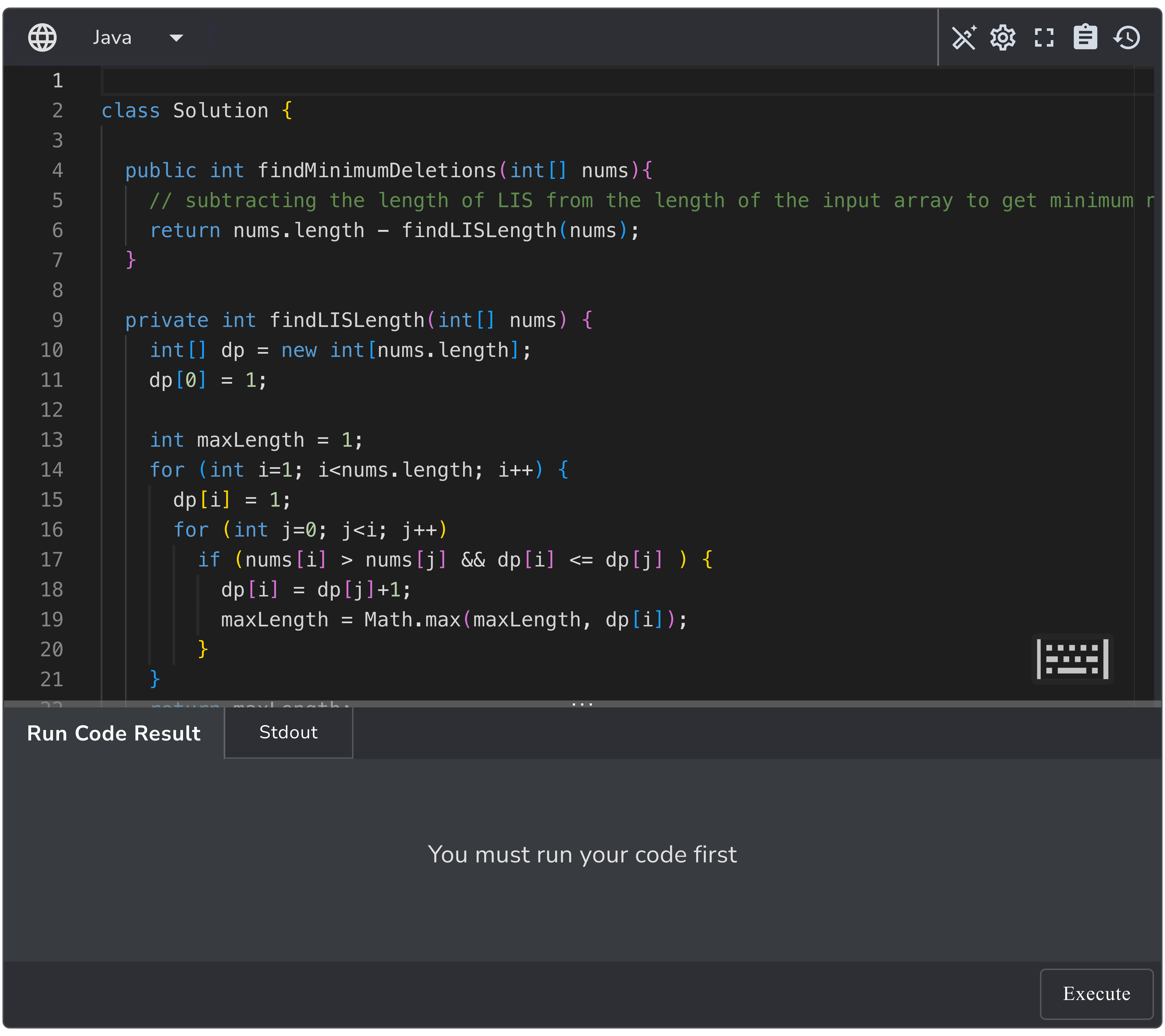
A basic brute-force solution could be to try deleting all combinations of elements, one by one, and checking if that makes the subsequence sorted.

Alternately, we can convert this problem into a `"Longest Increasing Subsequence" (LIS)` problem. As we know that LIS will give us the length of the longest increasing subsequence (in the sorted order!), which means that the elements which are not part of the LIS should be removed to make the sequence sorted. This is exactly what we need. So we'll get our solution by subtracting the length of LIS from the length of the input array: `Length-of-input-array - LIS()`

Let's jump directly to the bottom-up dynamic programming solution.

Bottom-up Dynamic Programming

Here is the code for our bottom-up dynamic programming approach:



```java

class Solution {

  public int findMinimumDeletions(int[] nums){
    // subtracting the length of LIS from the length of the input array to get minimum r
    return nums.length - findLISLength(nums);
  }

  private int findLISLength(int[] nums) {
    int[] dp = new int[nums.length];
    dp[0] = 1;

    int maxLength = 1;
    for (int i=1; i<nums.length; i++) {
      dp[i] = 1;
      for (int j=0; j<i; j++)
        if (nums[i] > nums[j] && dp[i] <= dp[j] ) {
          dp[i] = dp[j]+1;
          maxLength = Math.max(maxLength, dp[i]);
        }
    }
```

You must run your code first

The time complexity of the above algorithm is $O(n^2)$ and the space complexity is $O(n)$.

← Previous
Minimum Deletions To Make A Sequence Sorted

Next →
Longest Repeating Subsequence

☐ Mark as Completed